

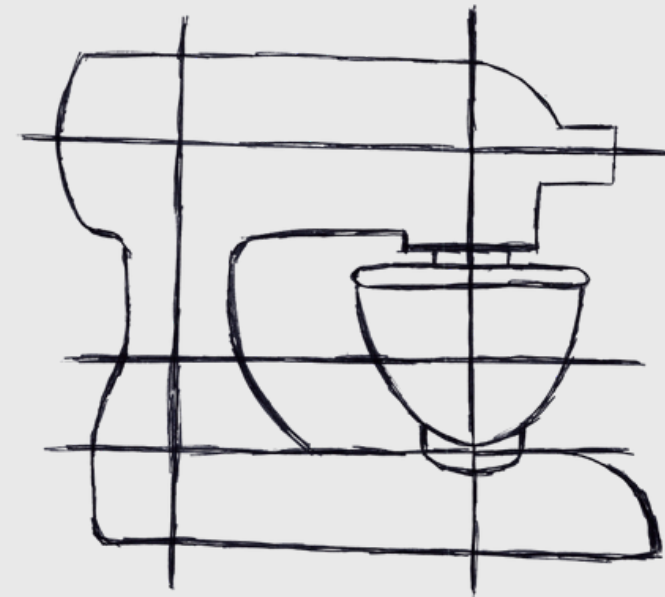
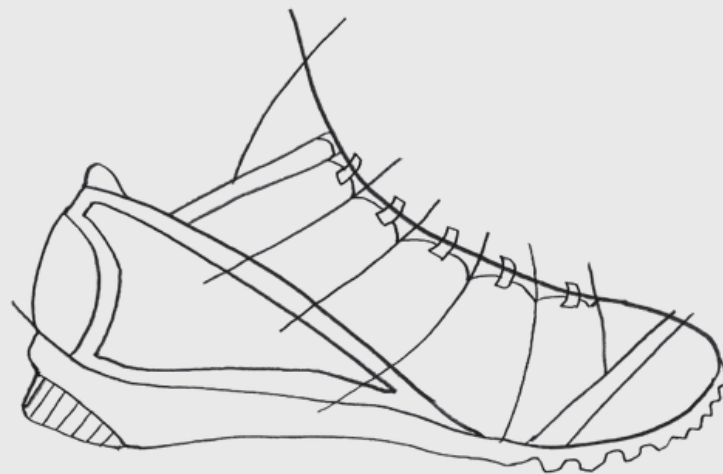
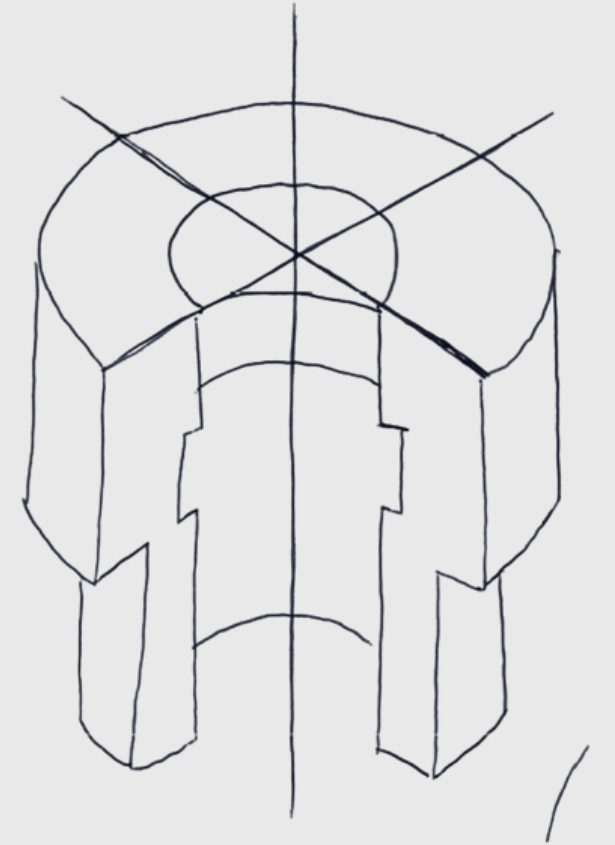
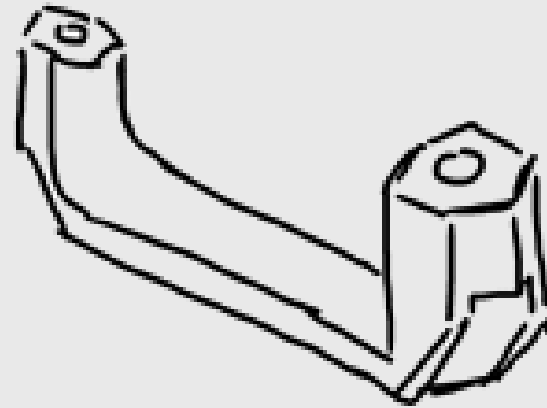


4IG08 - FINAL REPORT

Sketch Vectorisation

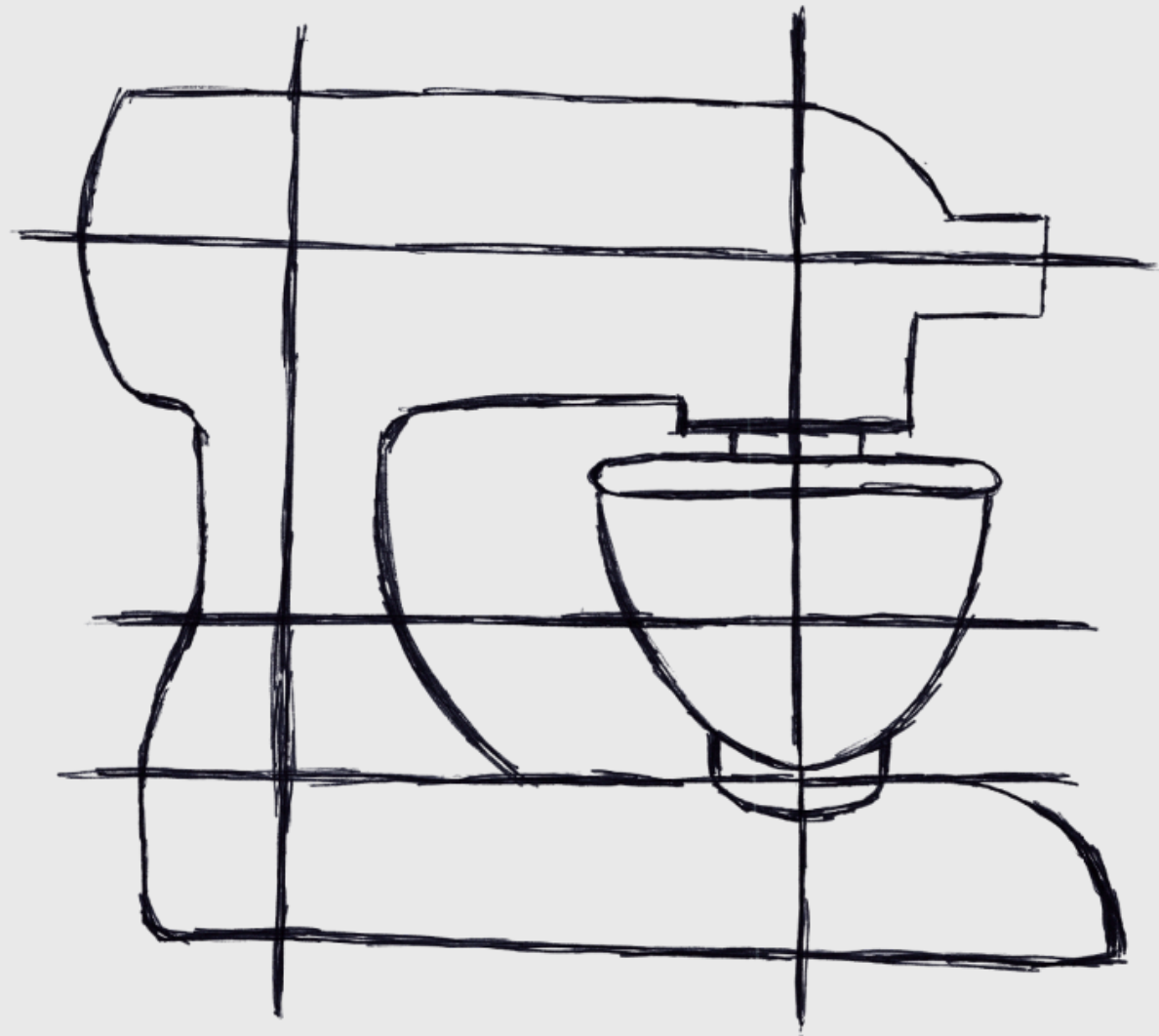
Mathéo Caroli / Antonin Peronnet / Lucas POTTIER

MOTIVATION

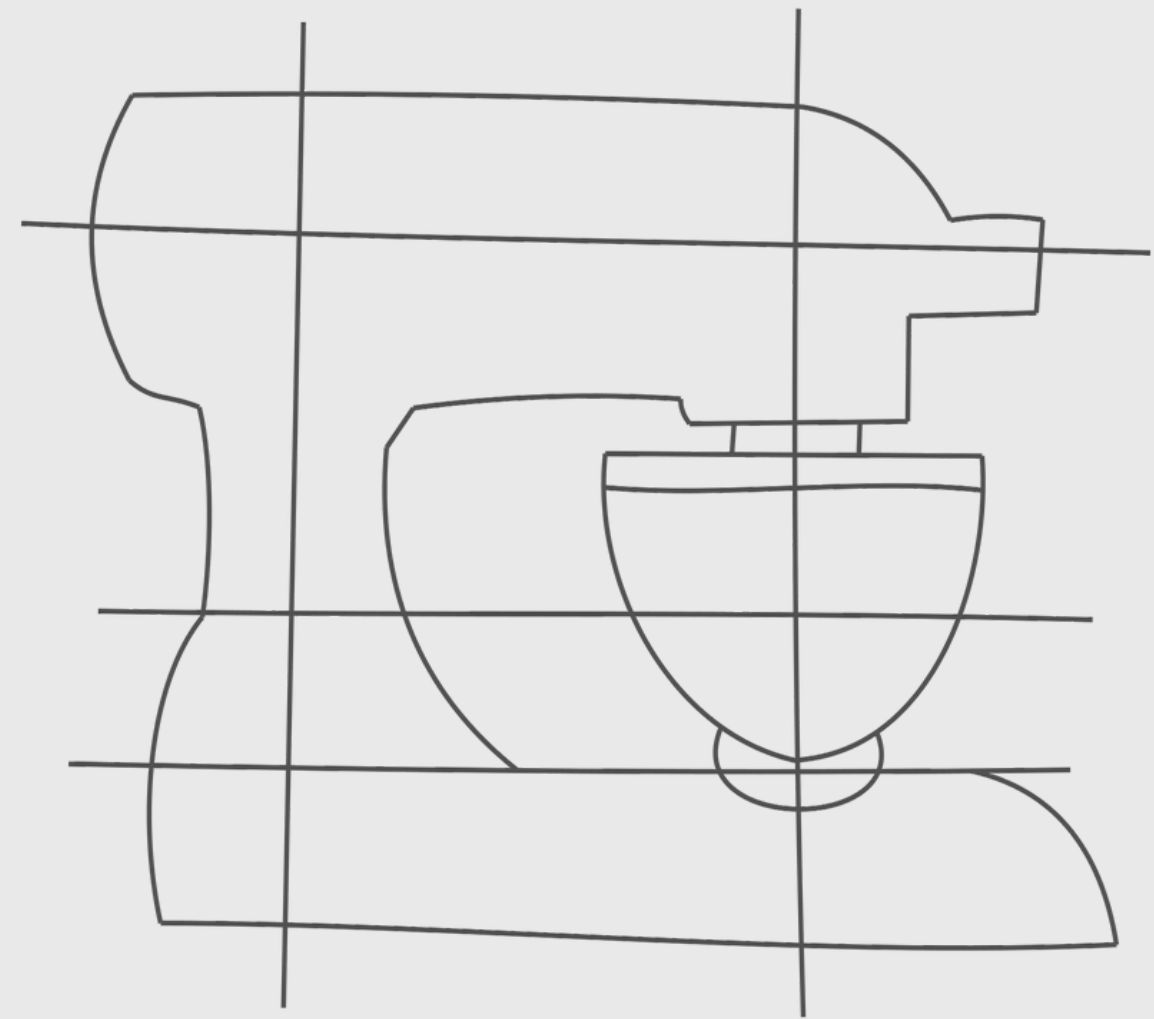


Hand drawn sketches examples

OBJECTIVE



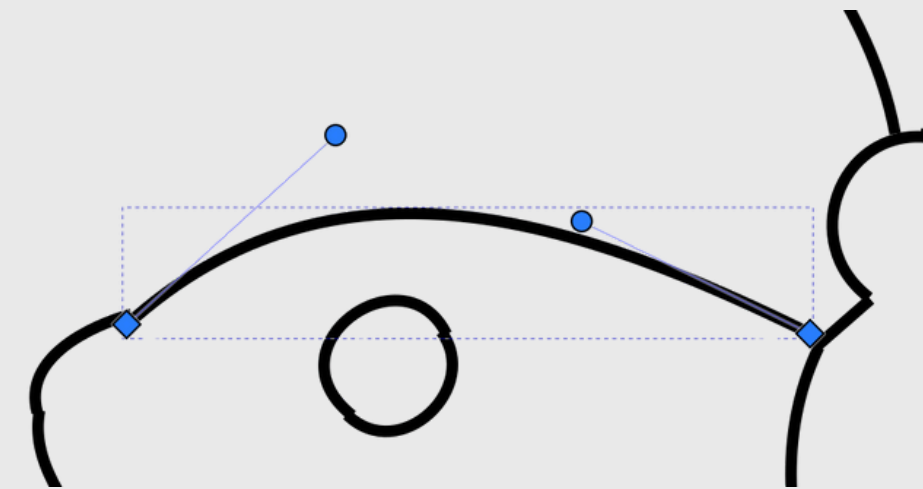
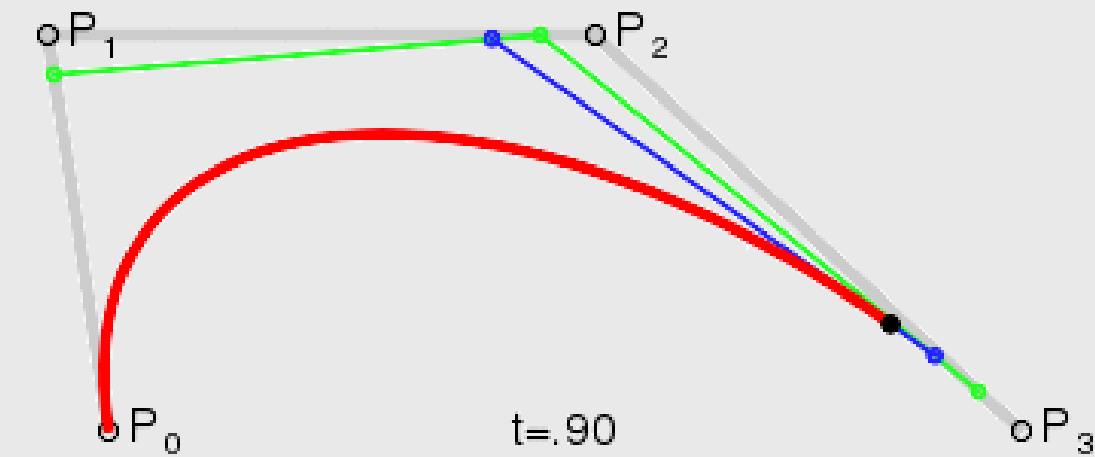
Hand drawn sketch



Vectorised sketch

BEZIER CURVES

- Ubiquitous in design software
- Basis of SVG
- Annoying to edit manually
- Pixel to bezier ? Yes !



SOURCES

- **Title :** *Fidelity vs. simplicity: a global approach to line drawing vectorization.*
- **Authors :** Jean-Dominique Favreau, Florent Lafarge, and Adrien Bousseau
- **Date :** 2016.

Fidelity vs. Simplicity: a Global Approach to Line Drawing Vectorization

Jean-Dominique Favreau

Florent Lafarge

Adrien Bousseau

Inria

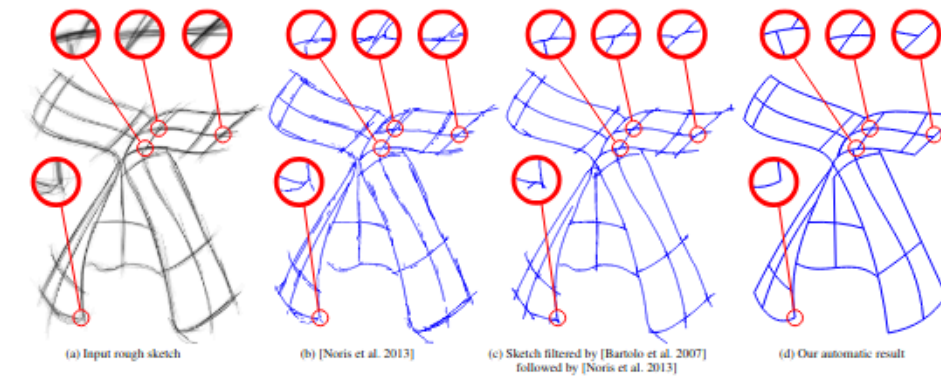


Figure 1: Rough sketches often contain overlapping strokes (a), which existing vectorization algorithms [Noris et al. 2013] represent as multiple curves (b). Pre-filtering the drawing with the method of Bartolo et al. [2007] improves the vectorization, but produces spurious curve segments at junctions (c). Since existing algorithms analyze junctions locally, they cannot recover the proper topology of these seemingly similar line configurations. By adopting a global formulation that optimizes for both fidelity to the input sketch and simplicity of the output curve network, our algorithm recovers proper topology while significantly reducing the overall number of curves and control points. Design sketch after Sori Yanagi's "Butterfly" stool.

Abstract

Vector drawing is a popular representation in graphic design because of the precision, compactness and editability offered by parametric curves. However, prior work on line drawing vectorization focused solely on faithfully capturing input bitmaps, and largely overlooked the problem of producing a compact and editable curve network. As a result, existing algorithms tend to produce overly-complex drawings composed of many short curves and control points, especially in the presence of thick or sketchy lines that yield spurious curves at junctions. We propose the first vectorization algorithm that explicitly balances fidelity to the input bitmap with simplicity of the output, as measured by the number of curves and their degree. By casting this trade-off as a global optimization, our algorithm generates few yet accurate curves, and also disambiguates curve topology at junctions by favoring the simplest interpretations overall. We demonstrate the robustness of our algorithm on a variety of drawings, sketchy cartoons and rough design sketches.

Keywords: Line drawing, sketch, vectorization, hypergraph, Bézier curves

Concepts: •Computing methodologies → Parametric curve and surface models; Reconstruction;

Publication rights licensed to ACM. ACM acknowledges that this contribution was authored or co-authored by an employee, contractor or affiliate of a national government. As such, the Government retains a nonexclusive, royalty-free right to publish or reproduce this article, or to allow others to do so, for Government purposes only.

1 Introduction

Vector drawings offer many advantages over bitmaps by representing graphical elements with parametric curves, such as Bézier splines. First, parametric curves are compact and resolution independent, while bitmap drawings require high storage to avoid aliasing. Second, parametric curves are defined by a small number of control points, which makes them easy to edit. The low dimensionality of the curves also contributes to the distinctive clean and sharp look of vector drawings, which contrasts with the noisy, rough look of bitmap sketches. Finally, parametric curves form the input of many advanced applications such as sketch-based modeling [Xu et al. 2014] and cartoon animation [Dalstein et al. 2015].

However, drawing parametric curves and adjusting their control points requires more precision and user interaction than freehand sketching, which is why many artists still prefer to draw in a bitmap form, either with pen and paper or with digital painting tools like Adobe Photoshop and Autodesk SketchBook. Our goal is to convert such rough, freehand bitmap sketches to clean vector drawings, keeping three main objectives in mind:

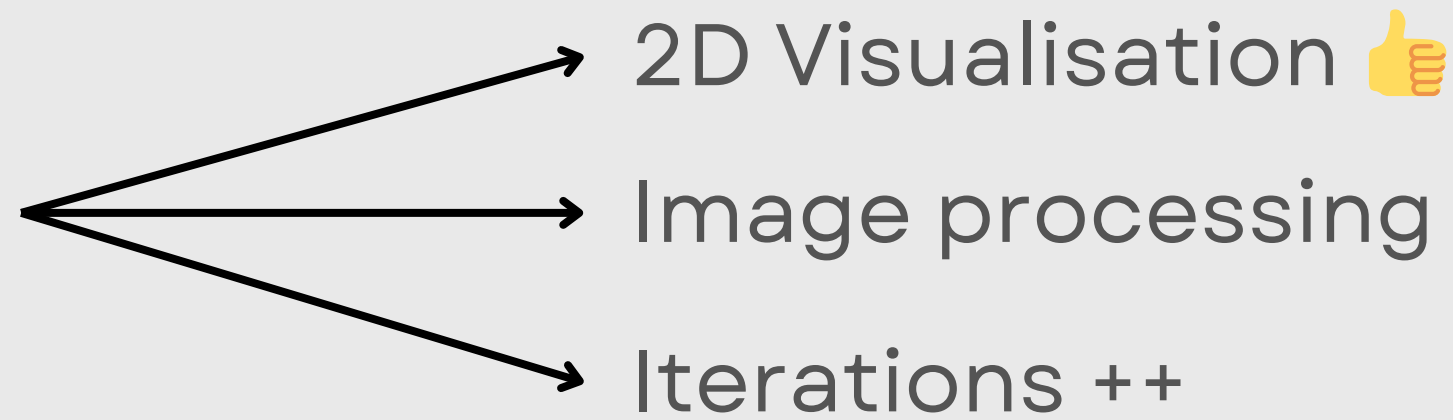
- **Fidelity.** The parametric curves should approximate well the input drawing.
- **Simplicity.** The result should be composed of a small number of curves with few control points to preserve the compactness and editability of vector graphics.

SIGGRAPH '16 Technical Paper, This is the author's version of the work
ISBN: 978-1-4503-4279-7/16/07
DOI: <http://dx.doi.org/10.1145/2897824.2925946>

IMPLEMENTATION CHOICES



Python



scikit-image
image processing in python

GROUP ORGANISATION

Documentation

3 weeks

- collective reading
- architecture design

Implementation

4 weeks

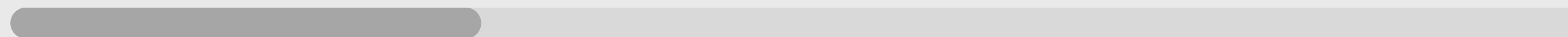
- atomic tasks
- **bottom up approach:**
create all pieces
and combine them

Evaluation

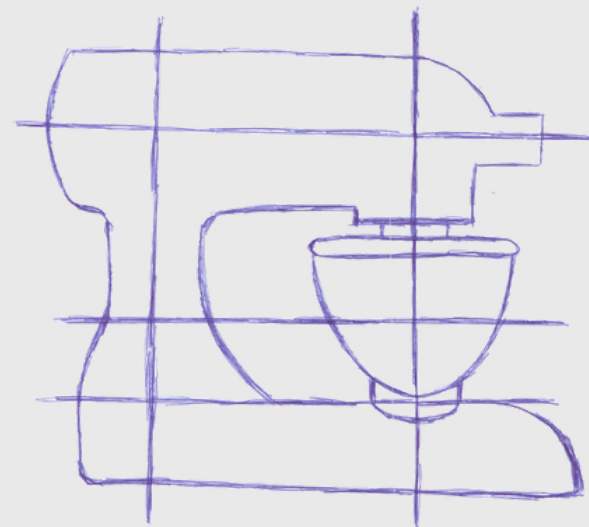
2 weeks

- putting all the code together
- writing the report
- comparing the results

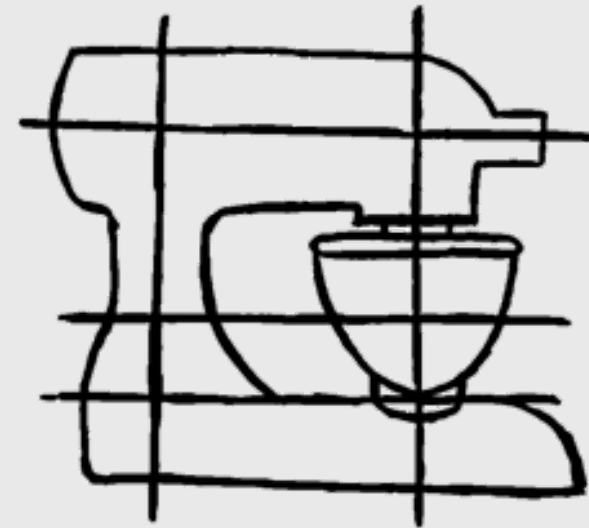
—————→ time



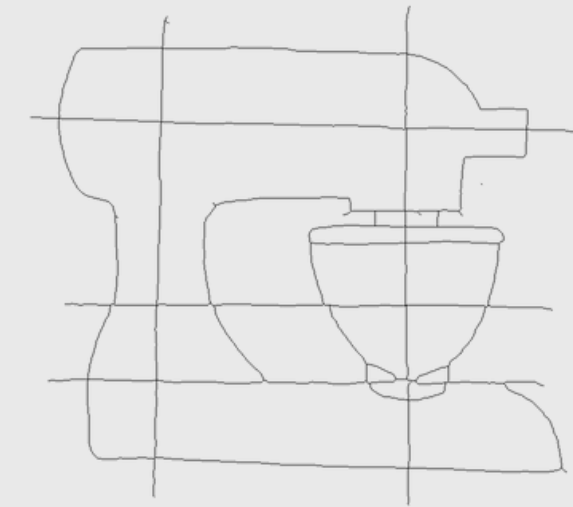
PIPELINE



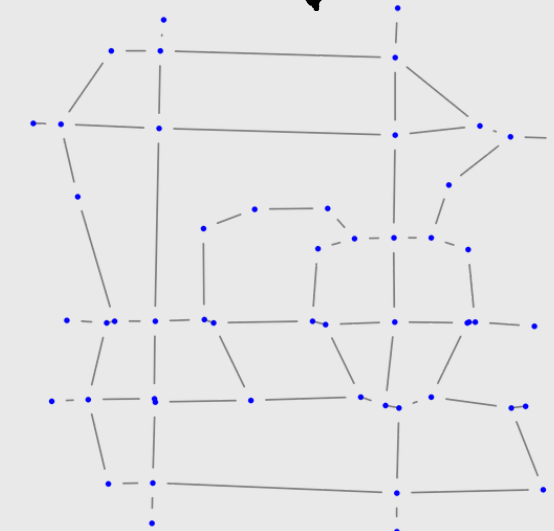
pixels



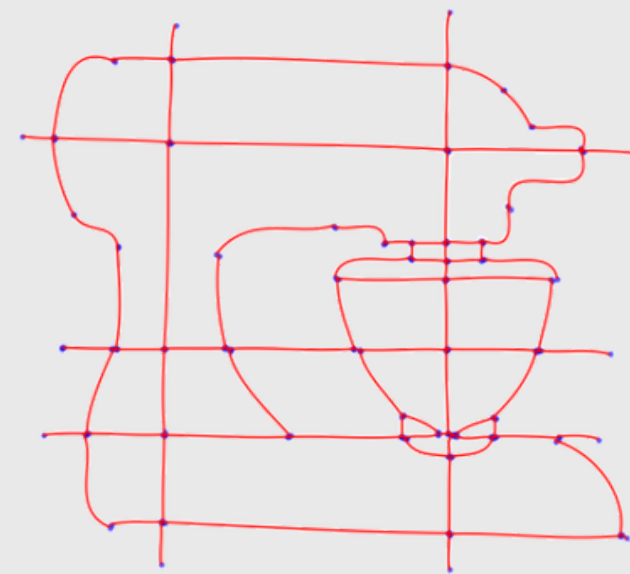
binary image



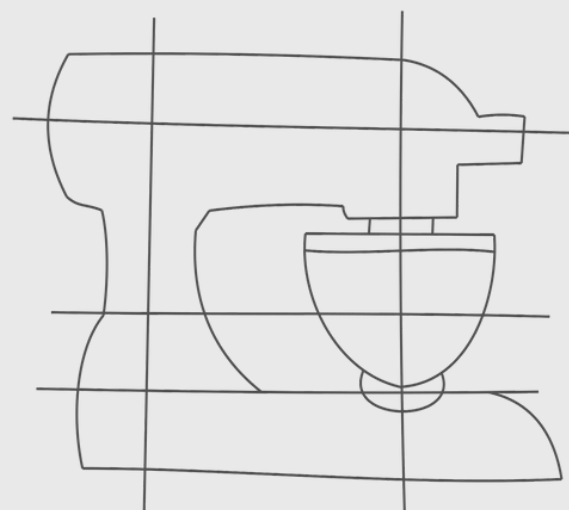
skeleton



topology



beziers

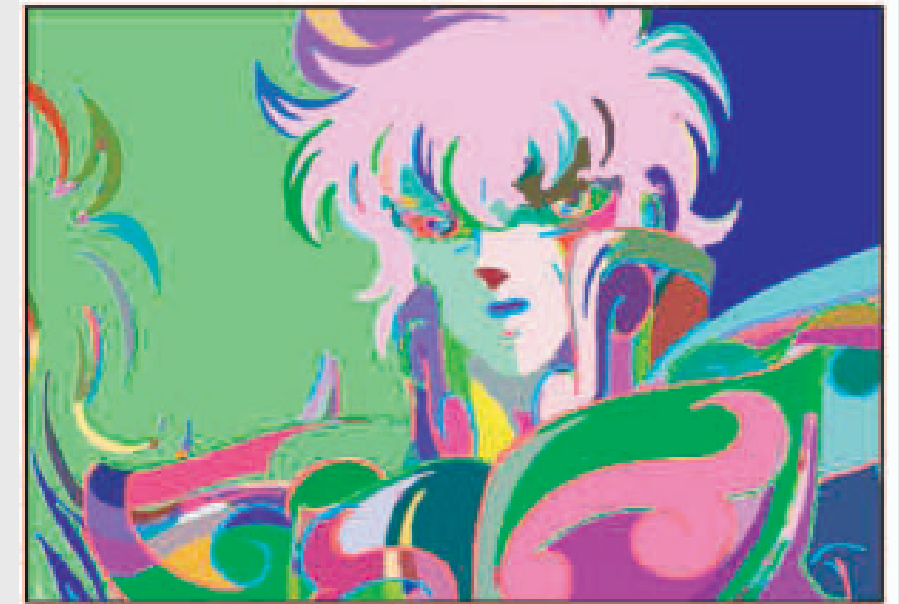


SVG

STEP 0: PREPROCESSING

Edge detection, segmentation, flood-filling, trapped ball, morphology ...

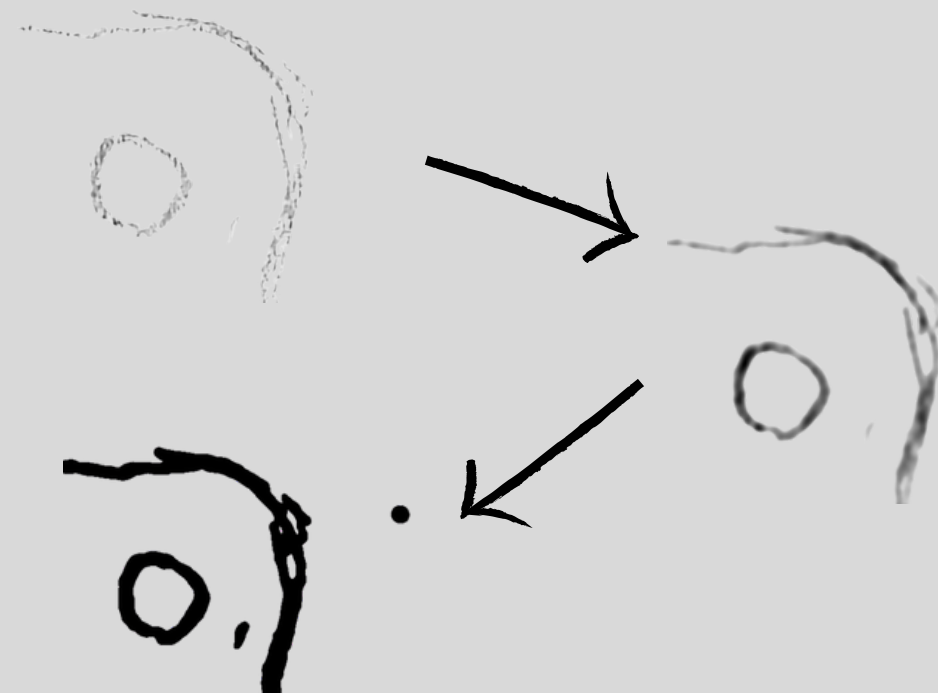
overkill ?



Innovation

1) Gaussian blurring

2) Local thresholding



Good enough
for most drawings
(not all ...)

STEP 1: SKELETON

Innovation

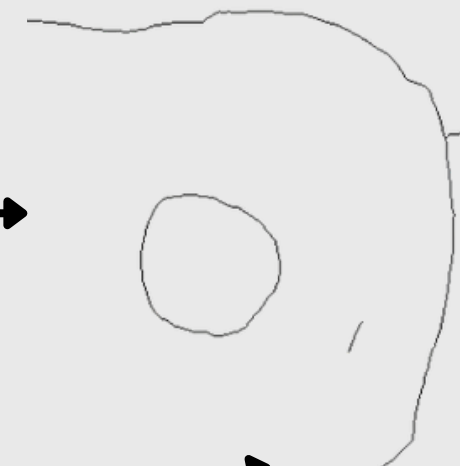
Morphological thinning for both closed and open curves

Dilation

Structuring element



Morphological closing
= Skeletonization

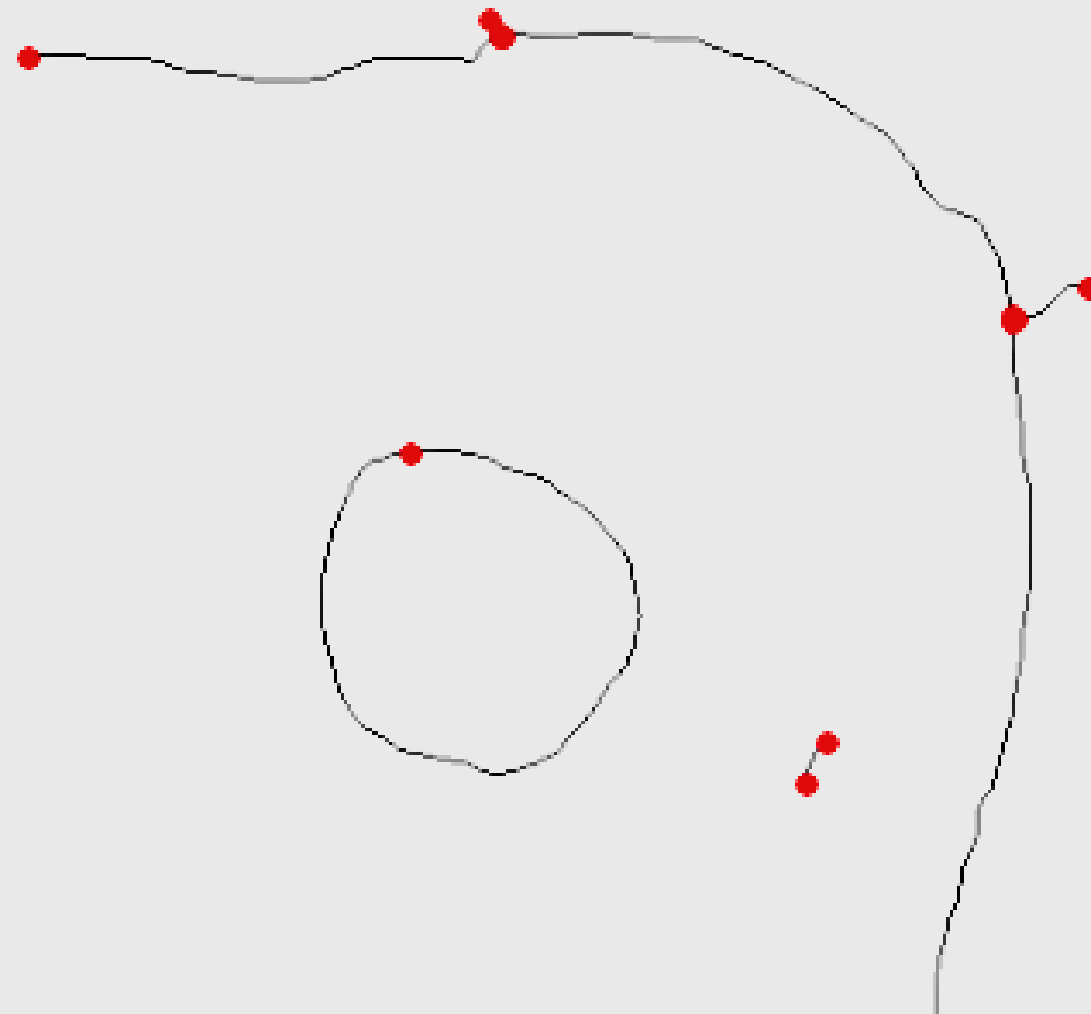


Dilation



Erosion =
morphological thinning

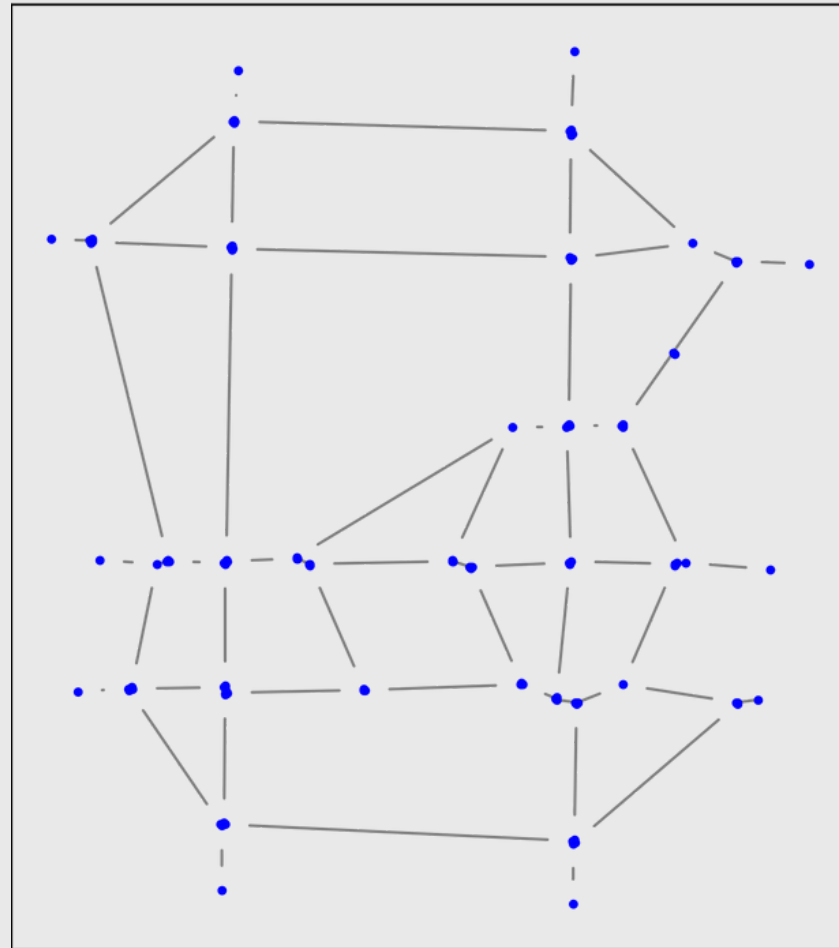
STEP 2: TOPOLOGY



Note: the paper was not explicit about the construction of the topology

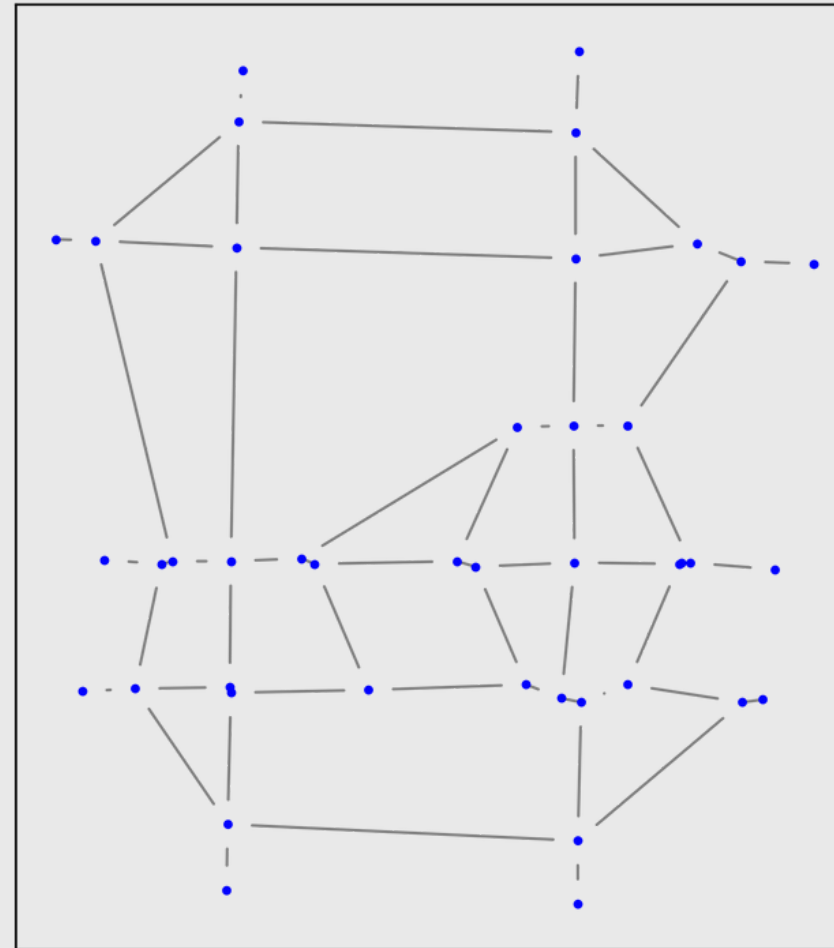
STEP 2: TOPOLOGY

Topology (naive)



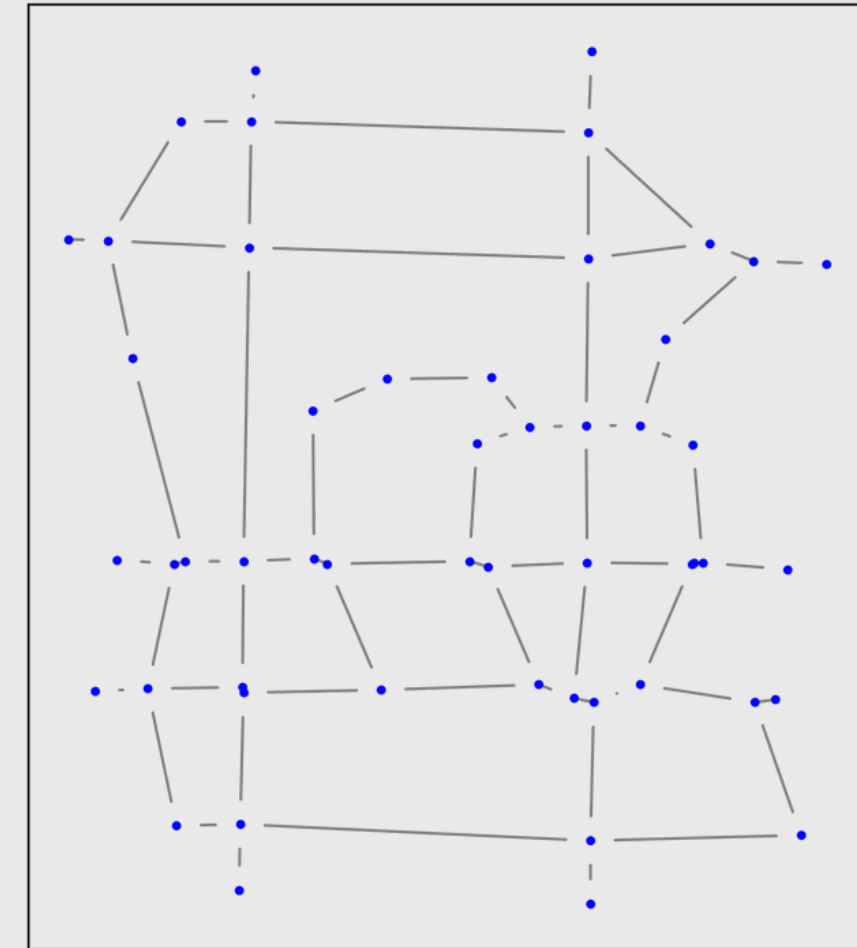
number of nodes: 138

Topology (with clustering)



number of nodes: 42

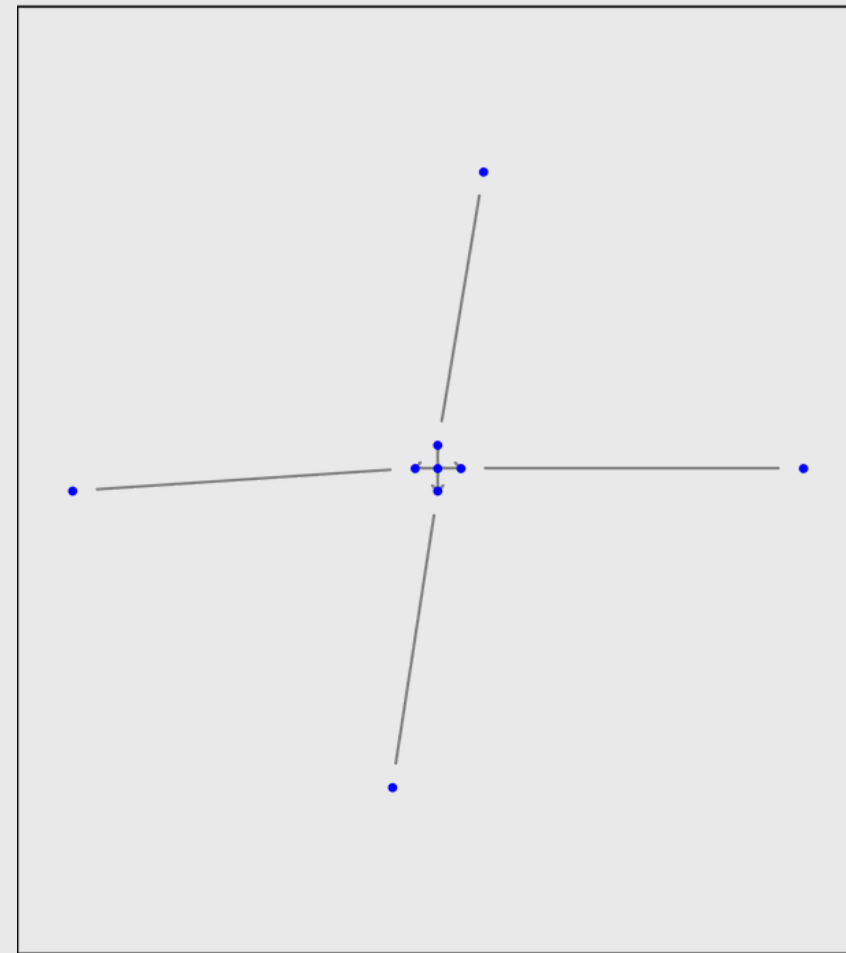
Topology (refined)



number of nodes: 52

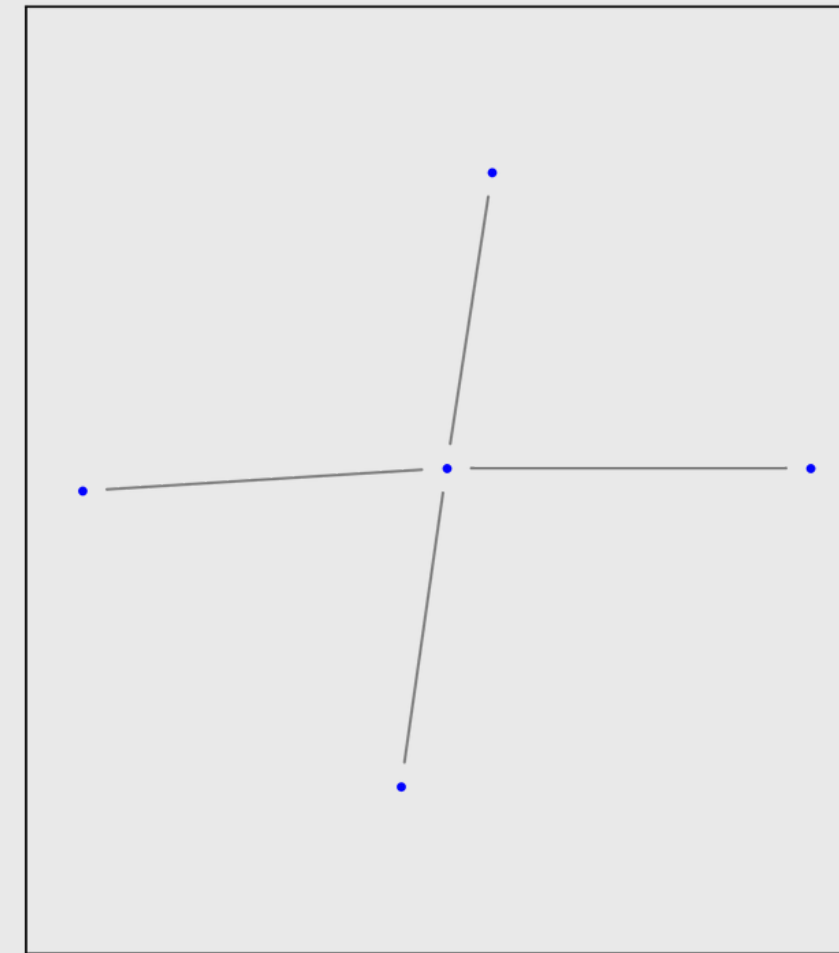
STEP 2: TOPOLOGY

Topology (naive)



number of nodes: 9

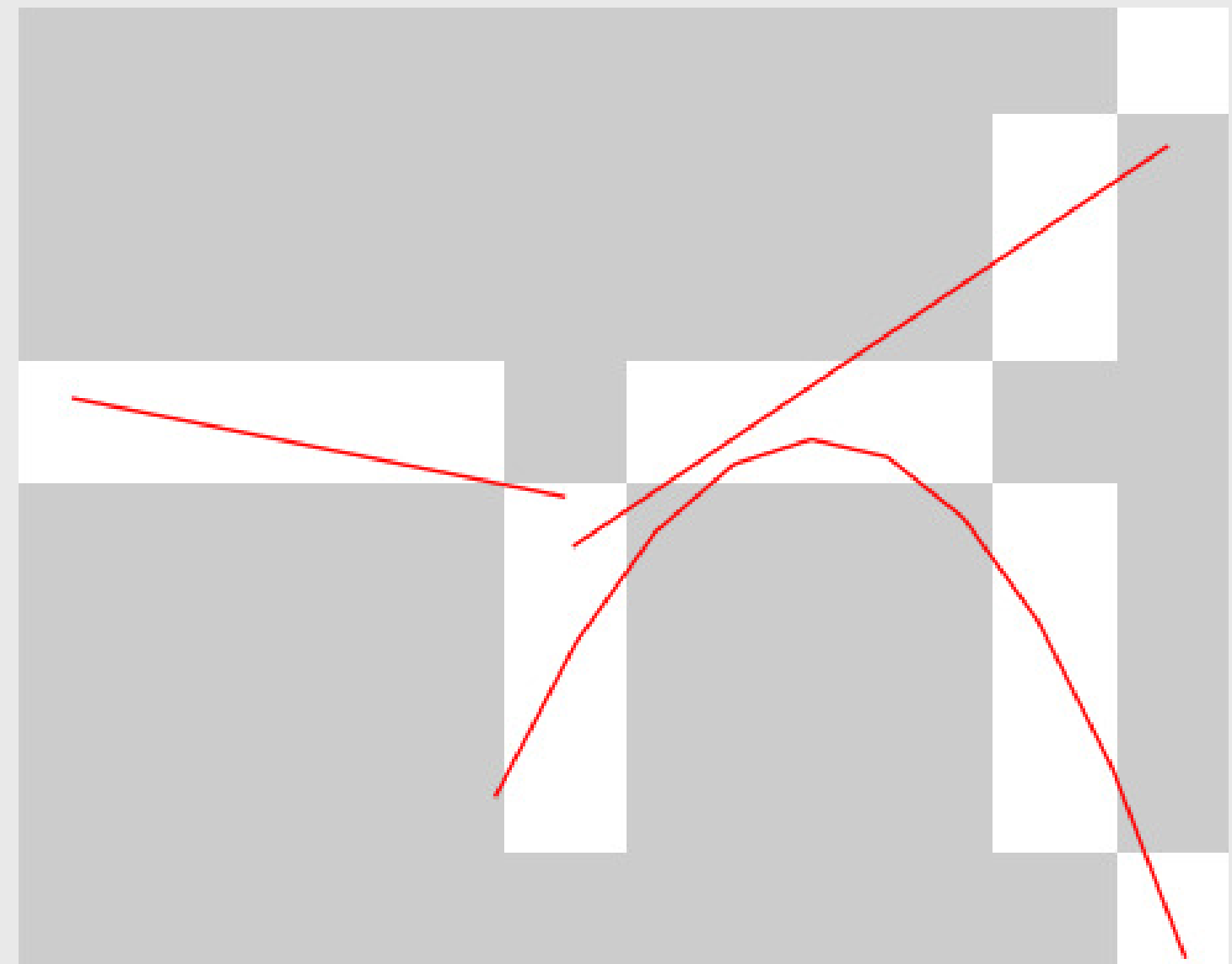
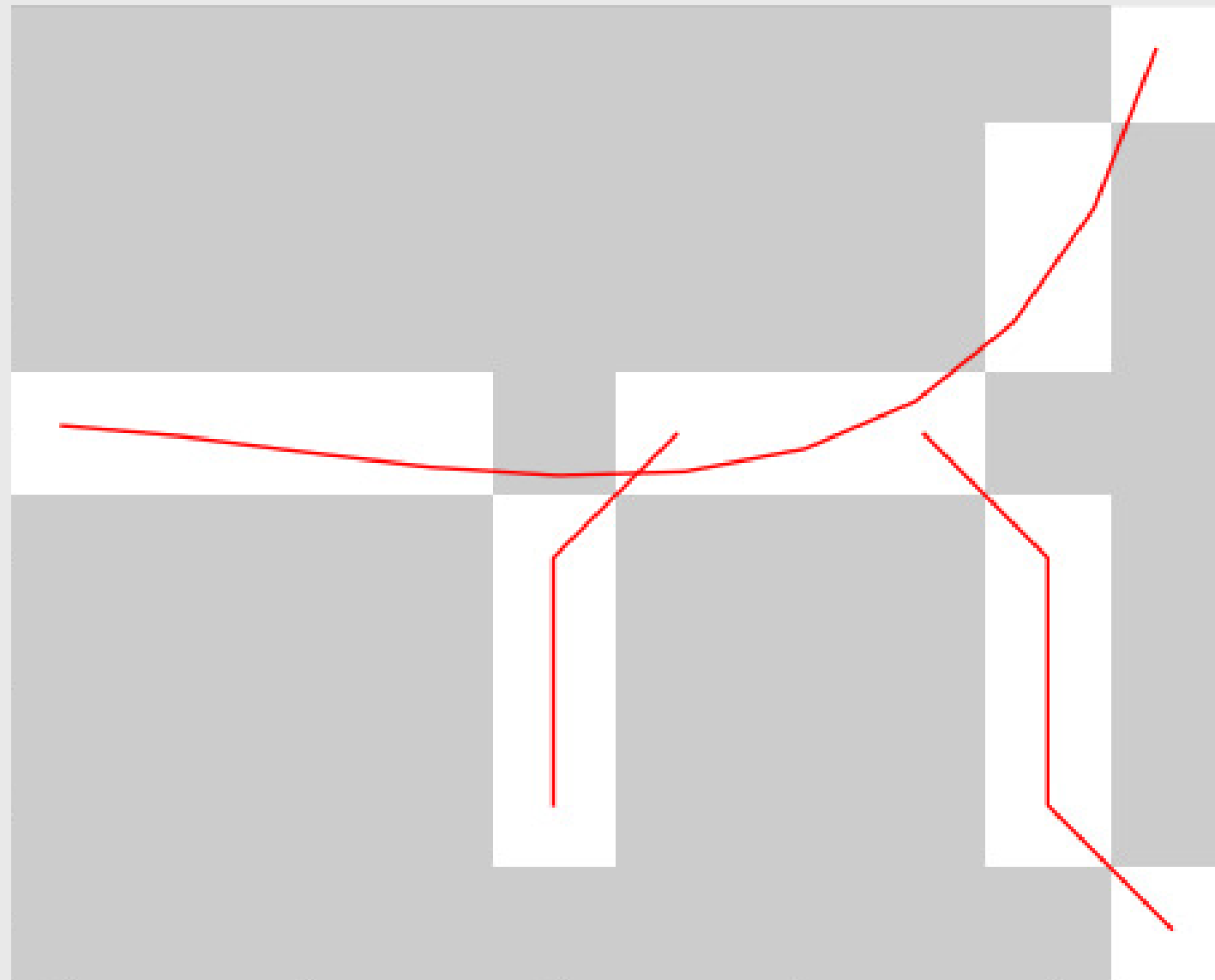
Topology (with clustering)



number of nodes: 5

STEP 3: OPTIMISATION

HOW DO WE DEFINE OPTIMALITY ?



STEP 3: OPTIMISATION

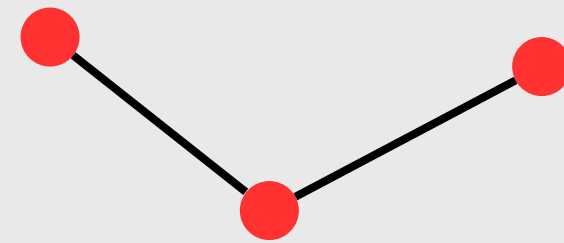
Fitting Bezier:
$$\epsilon(e) = \sum_{p \in S^e} \left(1 - \frac{w_p}{2}\right) \|B^e(t_p) - p\|_2^2$$

Global loss:
$$U(\mathbf{x}) = (1 - \lambda)U_{\text{fidelity}}(\mathbf{x}) + \lambda U_{\text{simplicity}}(\mathbf{x})$$

$$U_{\text{simplicity}}(\mathbf{x}) = \sum_{h \in H_{\mathbf{x}}} 1 + \mu \text{Deg}(B_{\mathbf{x}}^h)$$

$$U_{\text{fidelity}}(\mathbf{x}) = \sum_{h \in H_{\mathbf{x}}} \epsilon(h)$$

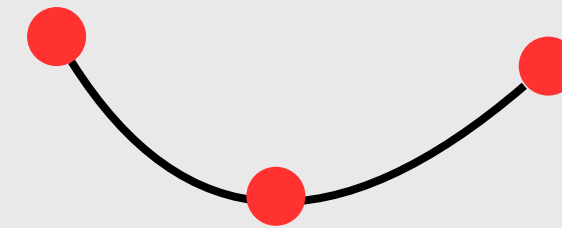
STEP 3: OPTIMISATION



score: 2

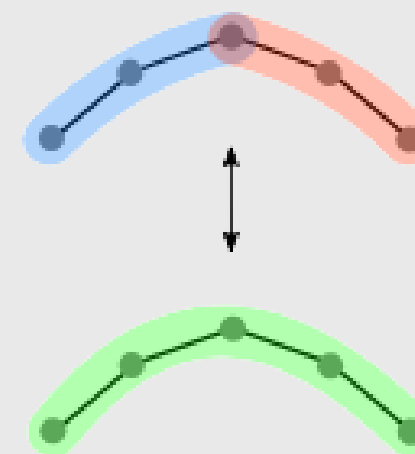


probability p

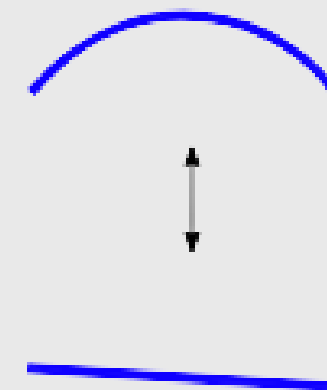


score: 3

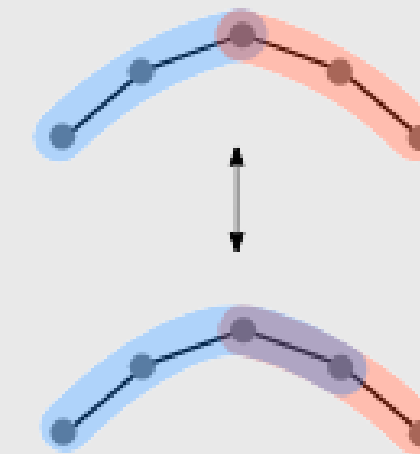
OPERATIONS



(a) Merge/Split



(b) Degree Switch



(c) Overlap/Dissociation

- Simulated annealing with geometric decrease

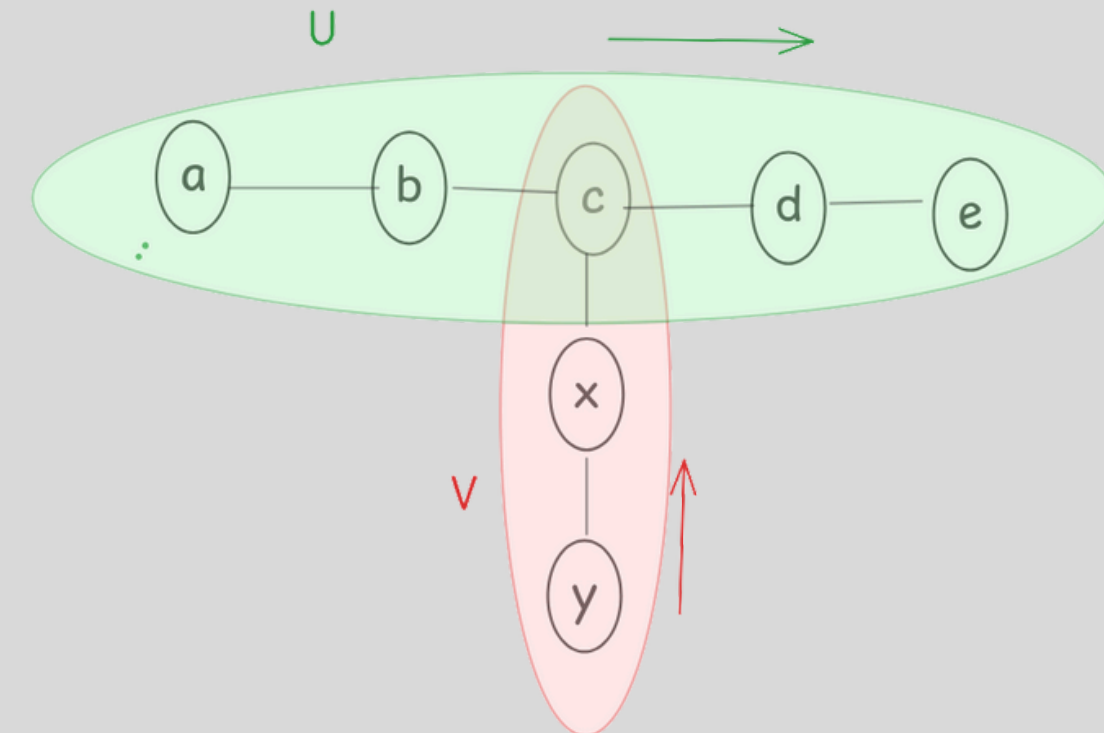
STEP 3: OPTIMISATION

Innovation

Derive the loss function for Bezier fitting:
We get a 4x4 linear equation problem
→ improved efficiency

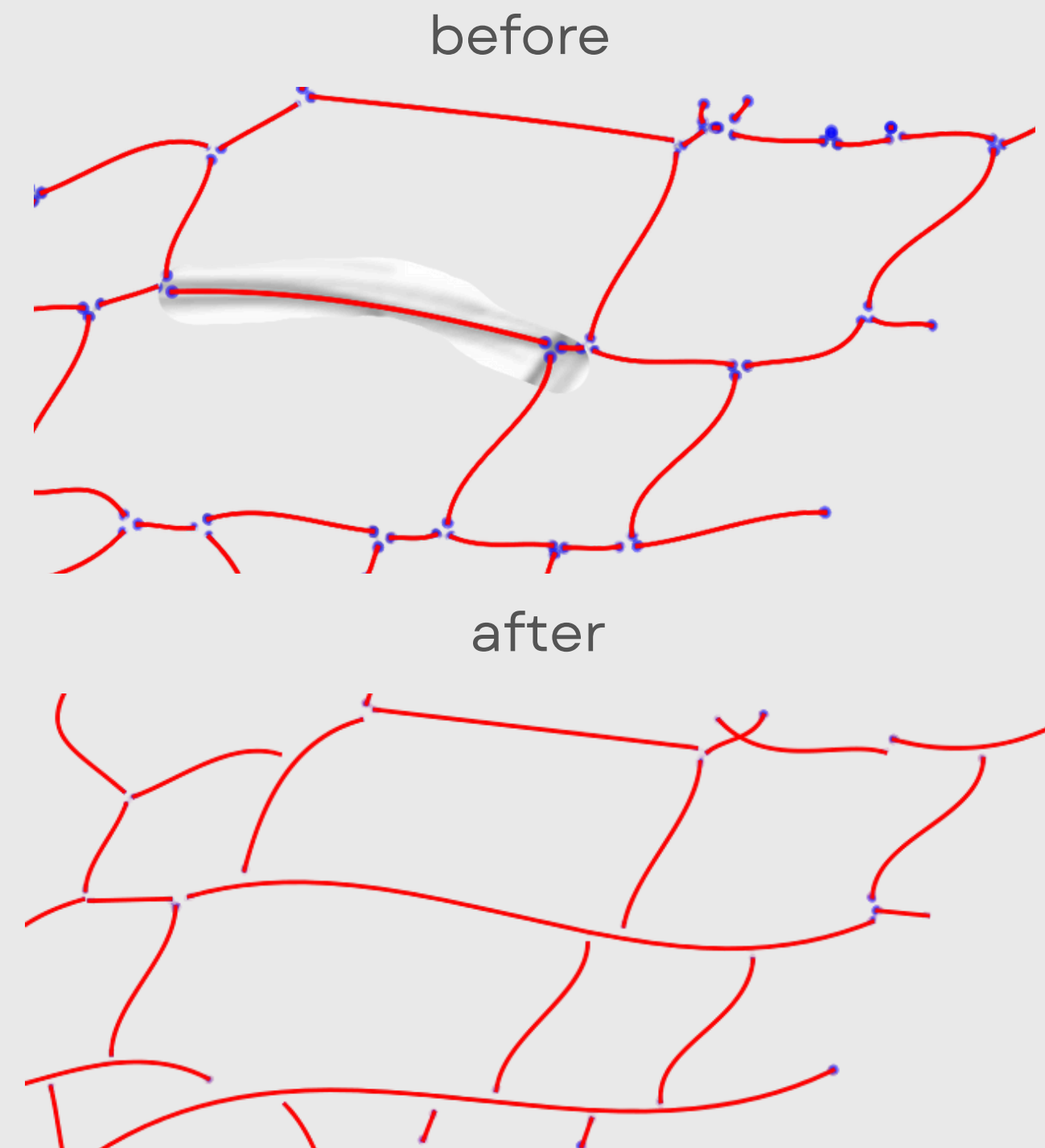
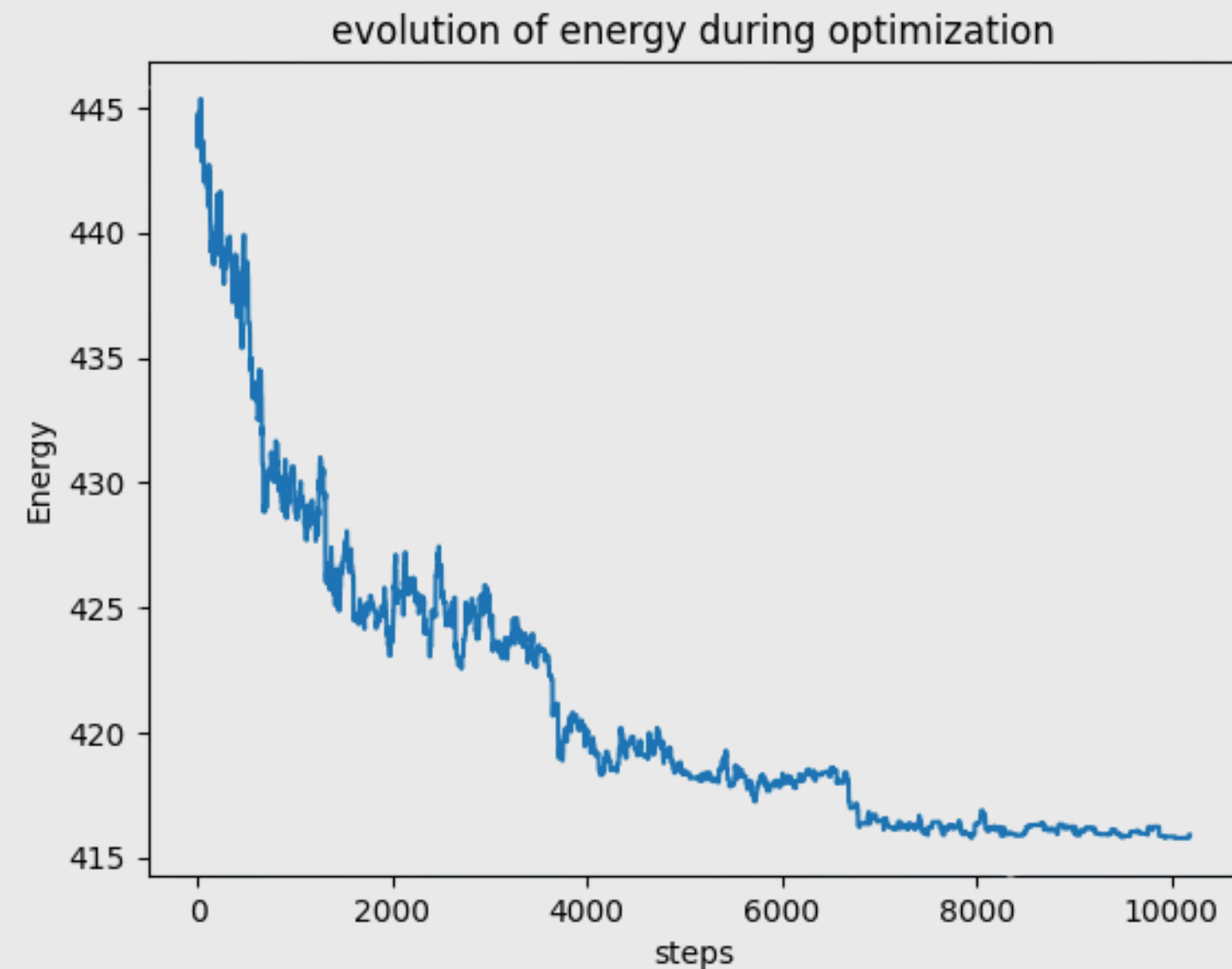
Innovation

consider **oriented** hyperedges,
and add a reverse operation
→ cleaner code



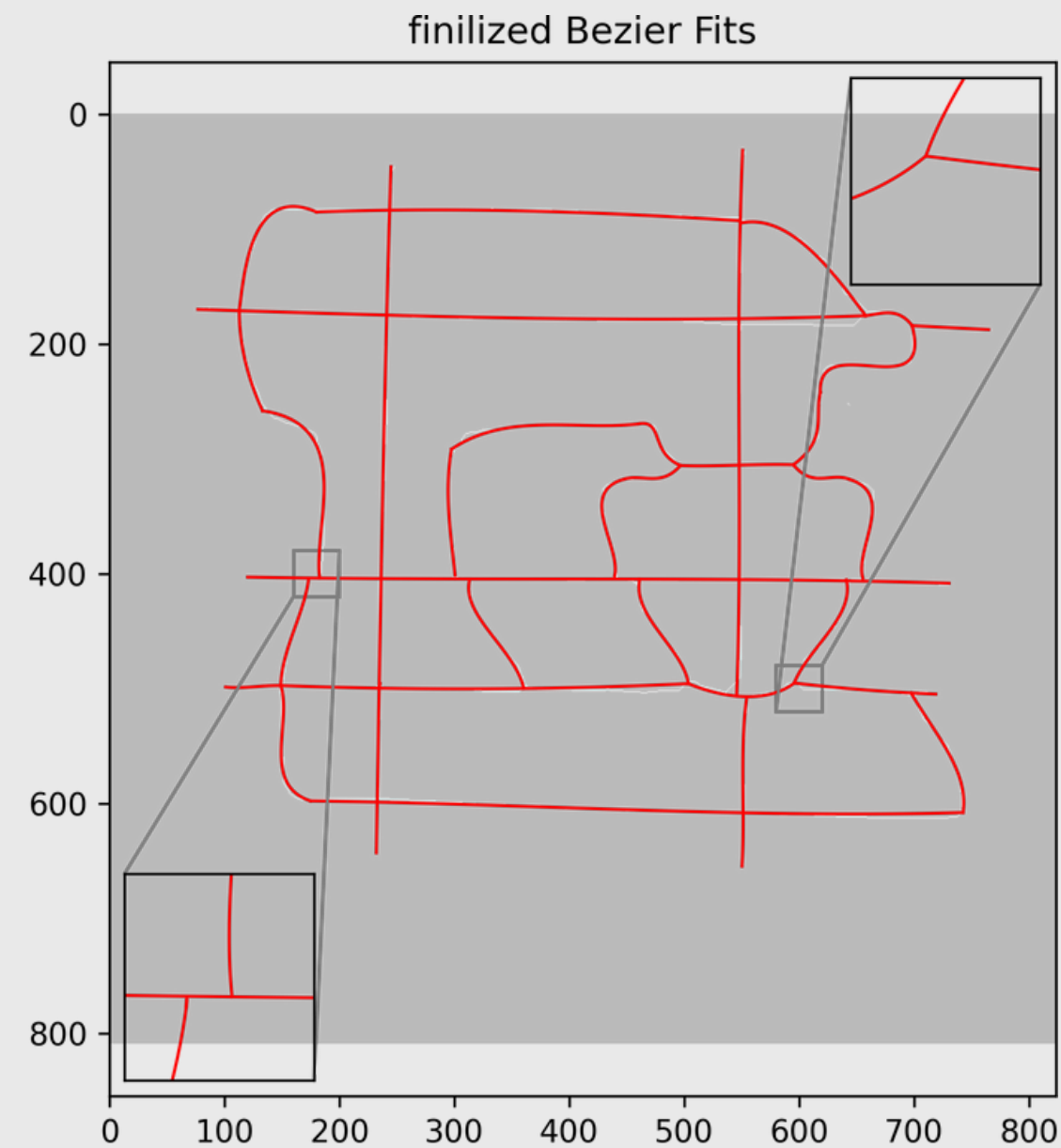
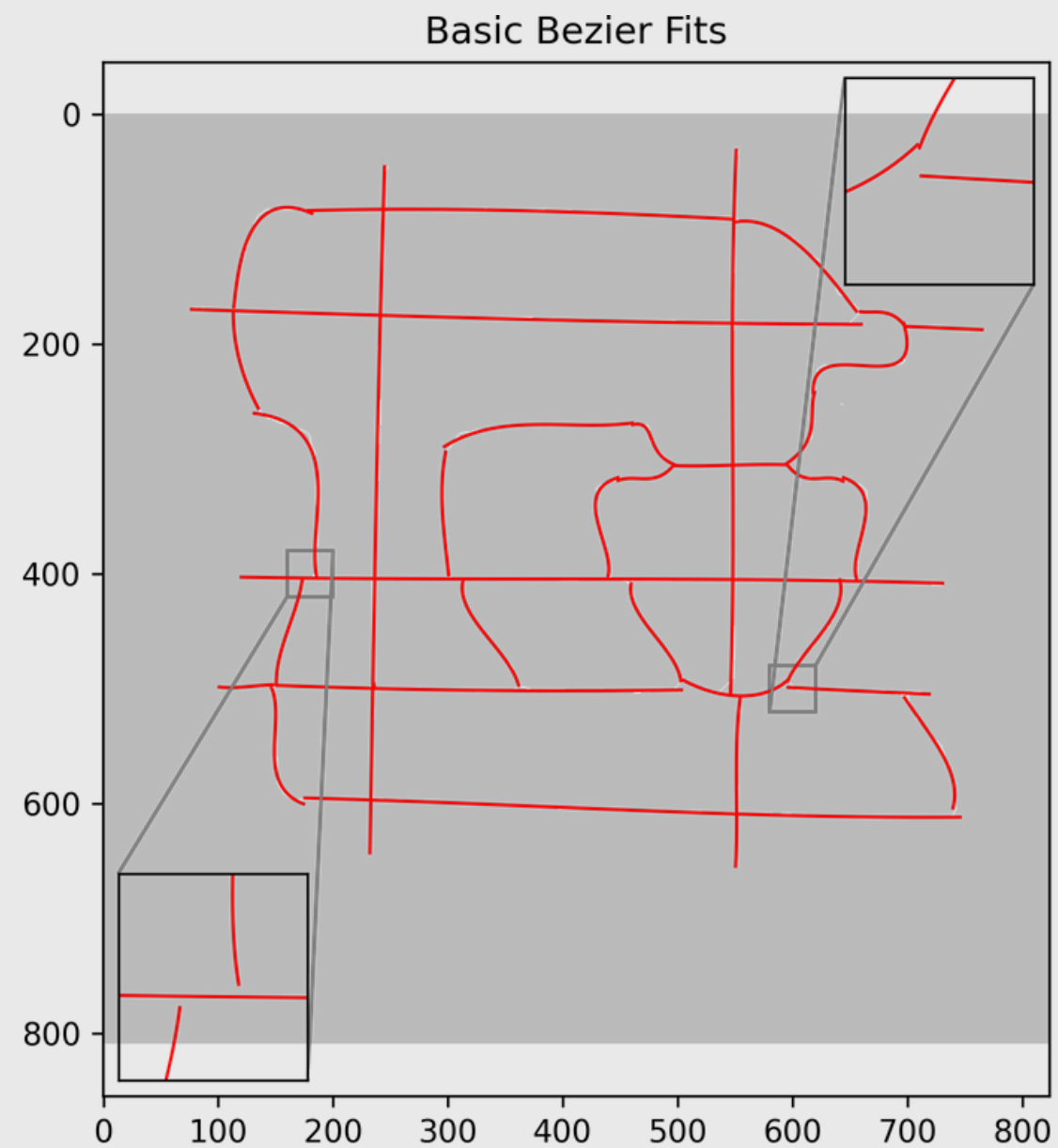
STEP 3: OPTIMISATION

STOCHASTIC EXPLORATION



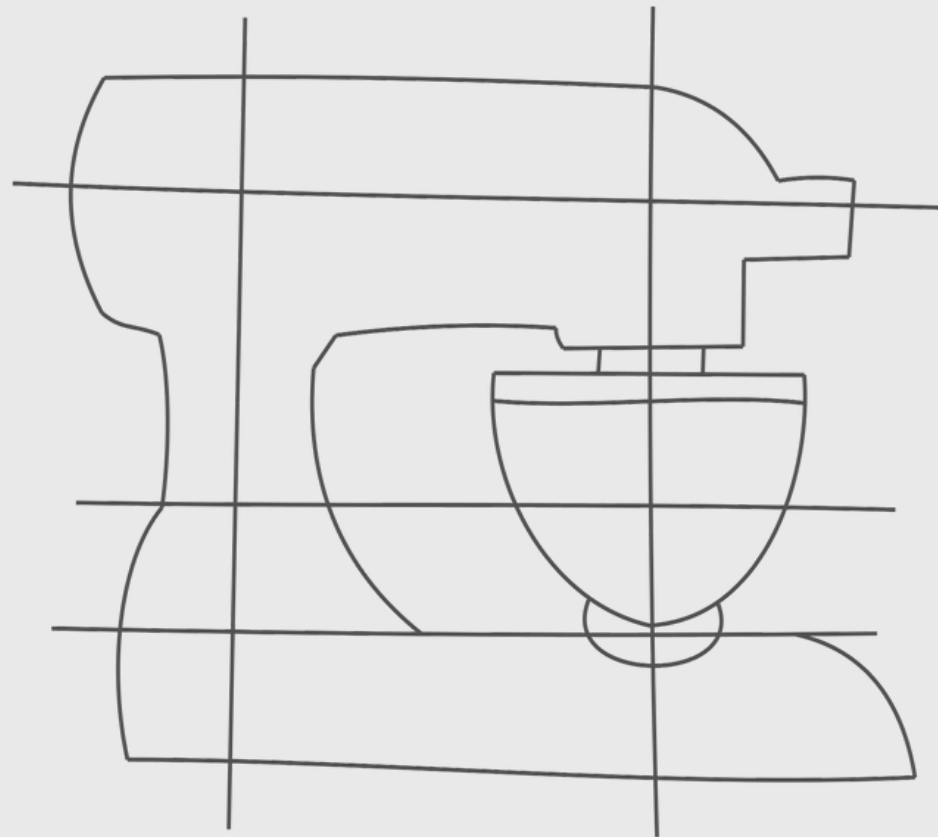
STEP 4: FINALISATION

JOINING BÉZIER CURVES

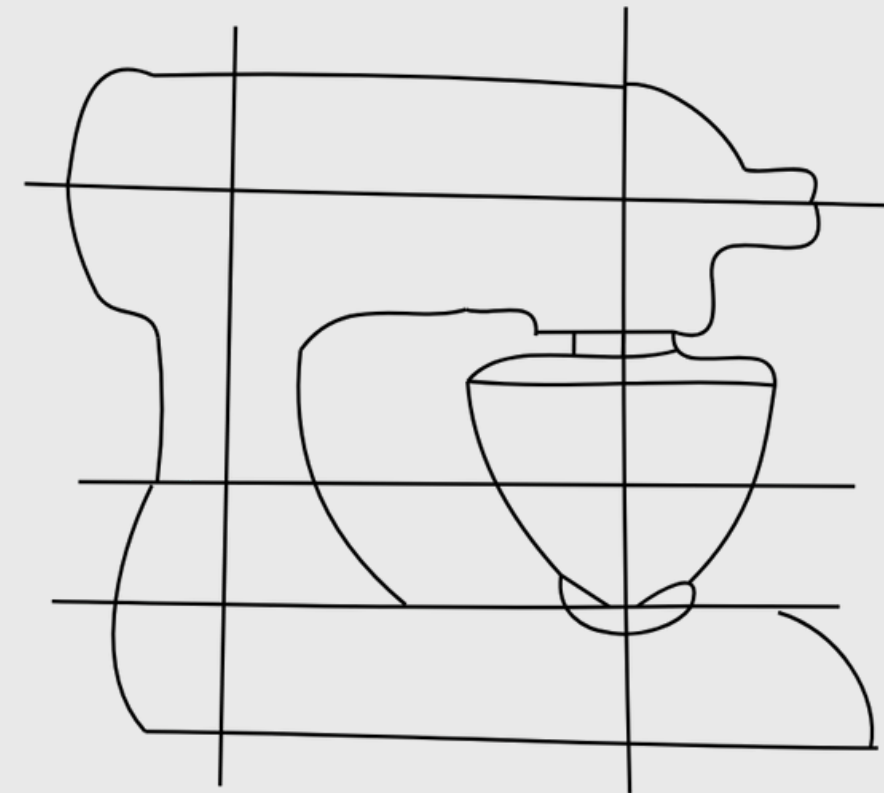


RESULTS

HOW DO WE COMPARE TO THE ORIGINAL PAPER ?



Their implementation



Our implementation

Possible explanations:

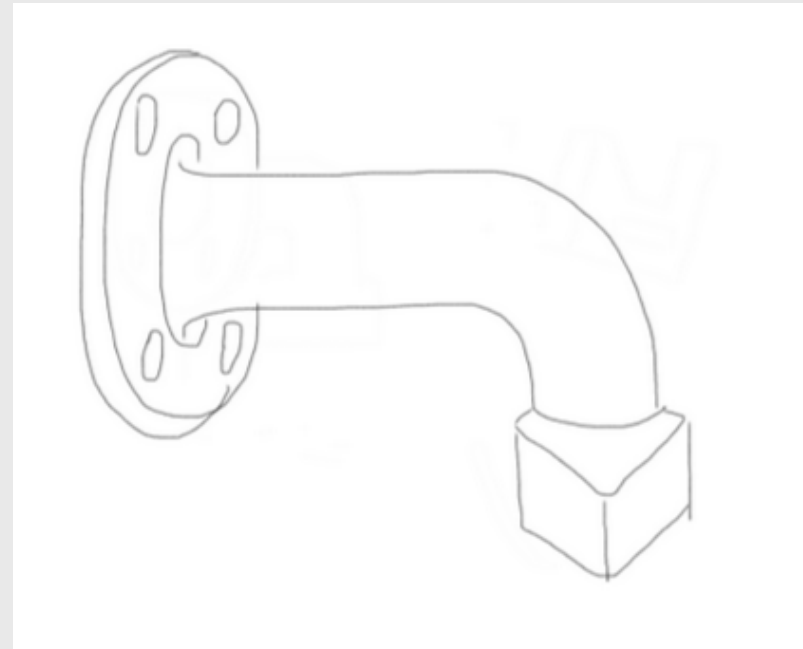
- region-based segmentation
- different logic for constructing topology

DISCUSSION

LIMITATIONS OF OUR IMPLEMENTATION



need trapball



Do not need traped ball

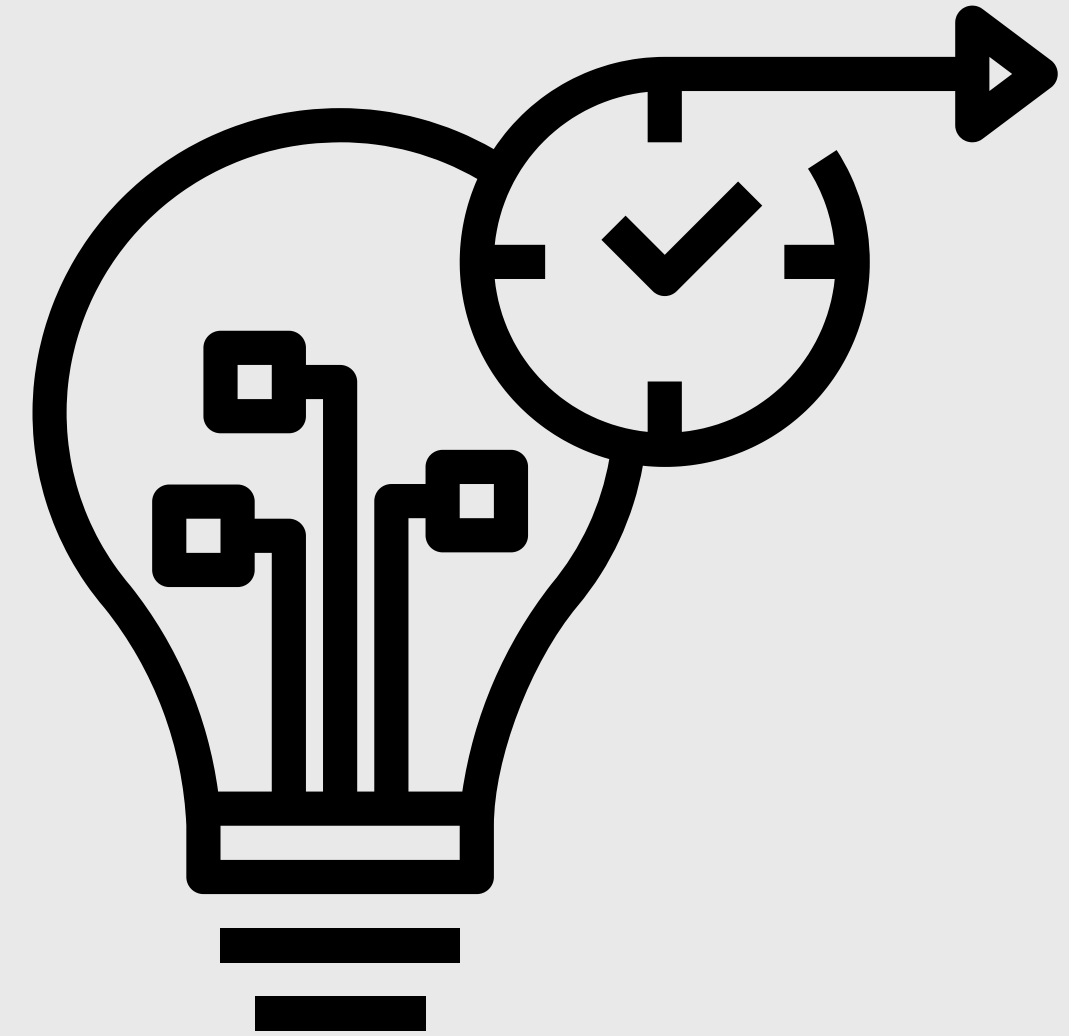
Paper lacks **reproducibility**:

- morphological thinning
- construction of topological graph
- perturbations
- junctions

POSSIBLE IMPROVEMENT

WHAT COULD WE DO NOW ?

- Support for color images
- Try other hyper-parameters
- User interactivity

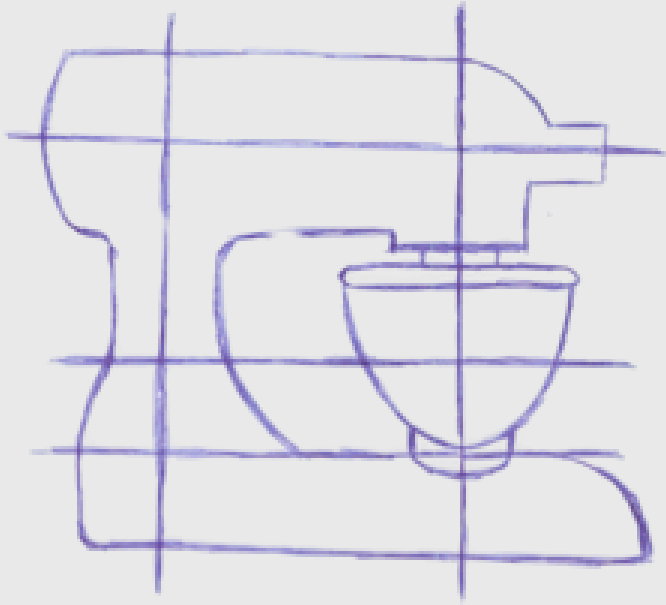




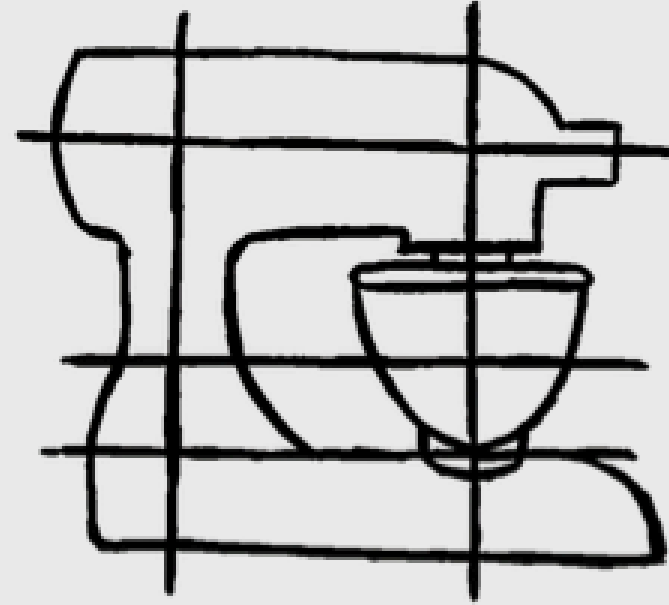
THANK YOU

ADDITIONAL MATERIAL

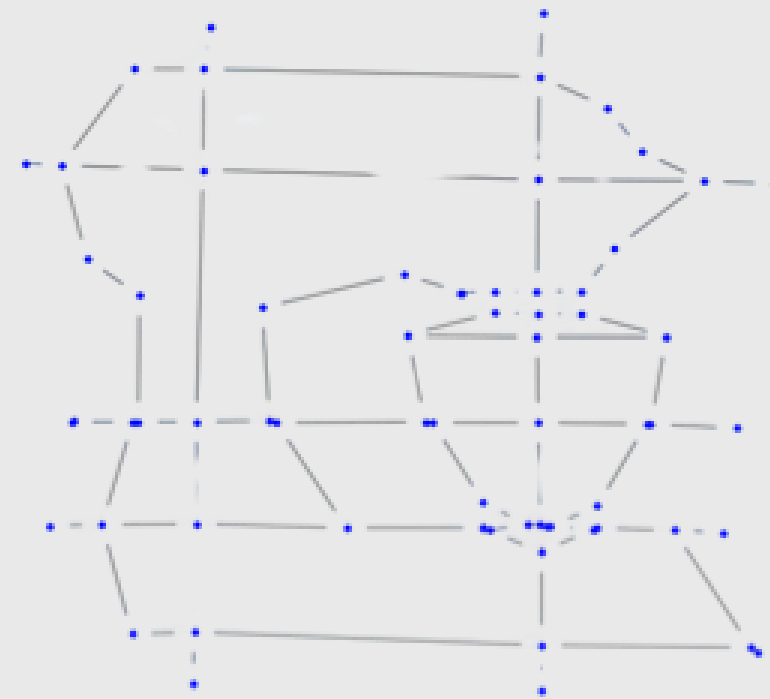
Original Image



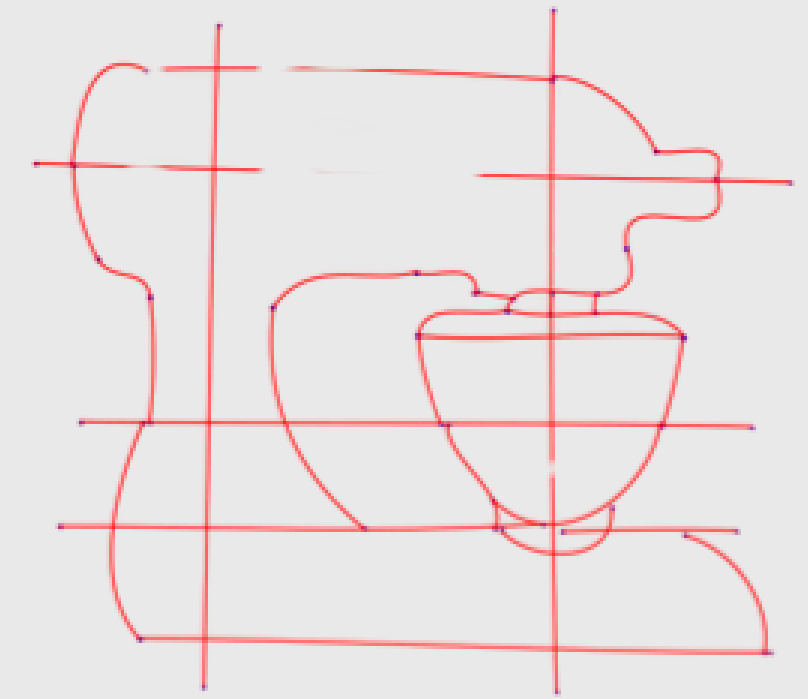
Binary Image



Refined Topology



Result



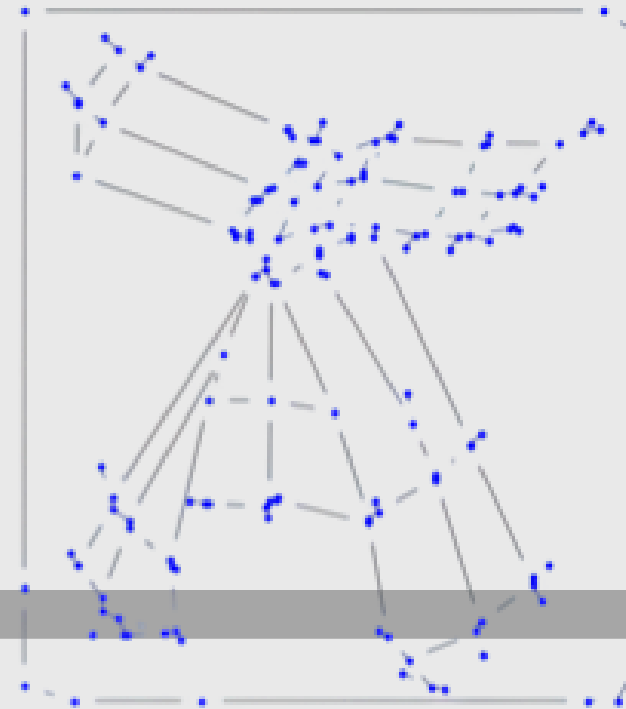
Original Image



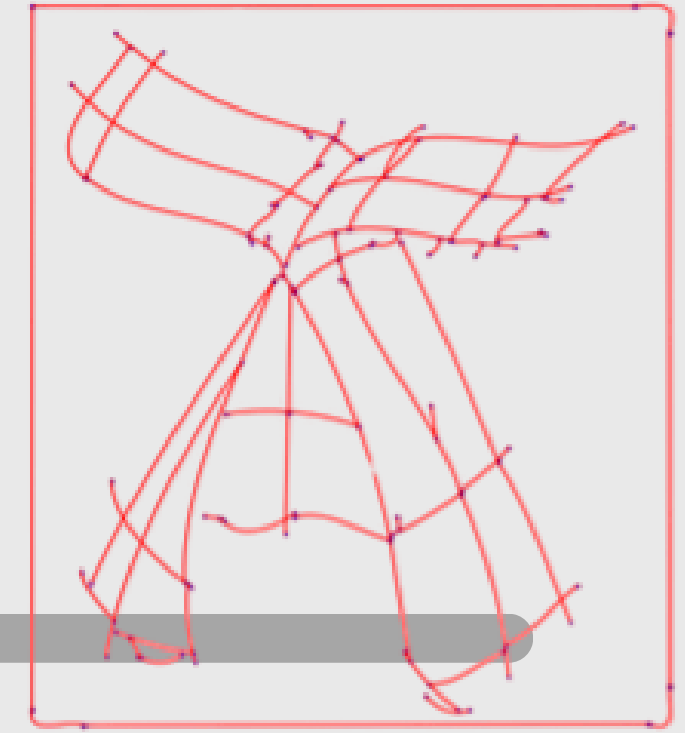
Binary Image



Refined Topology

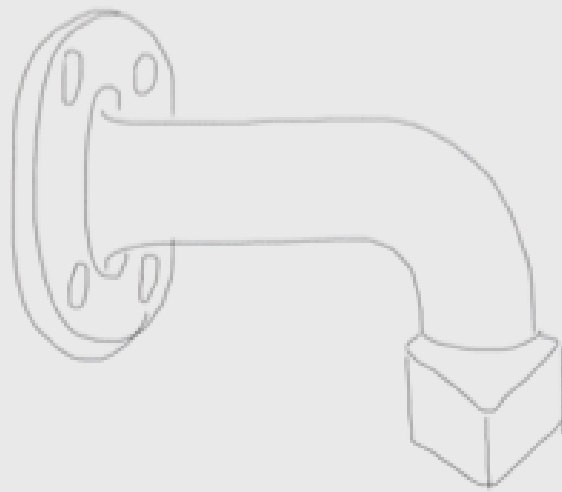


Result

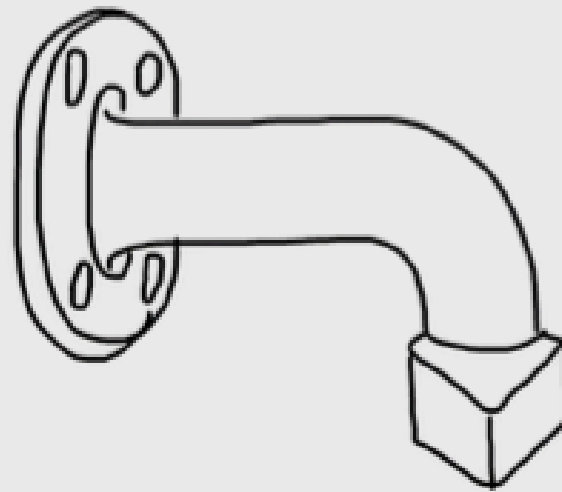


ADDITIONAL MATERIAL

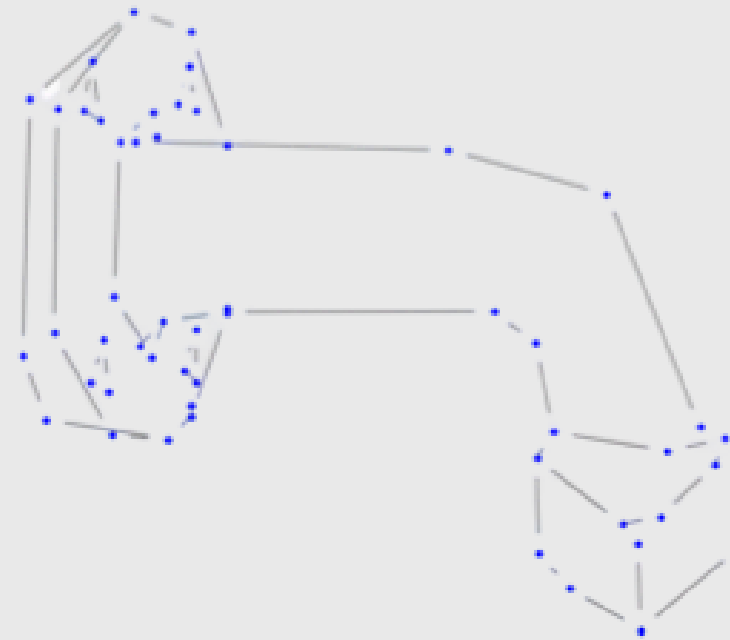
Original Image



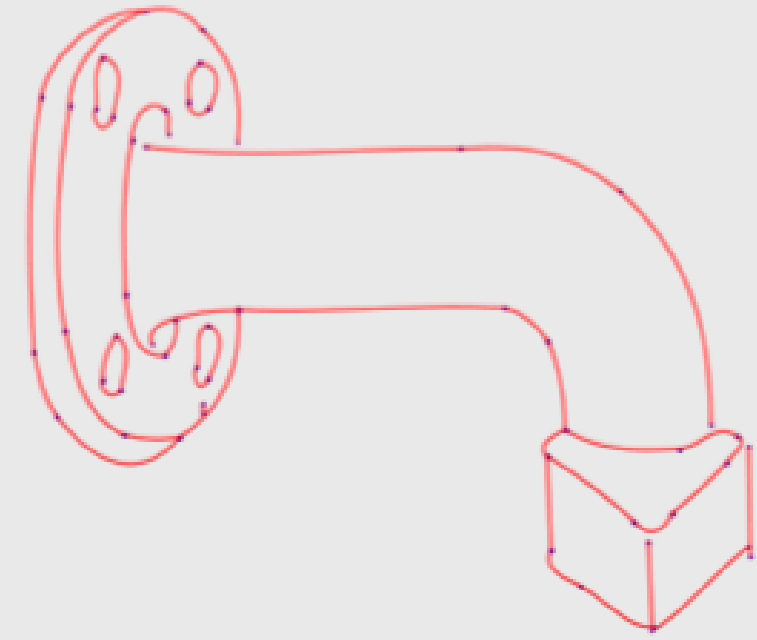
Binary Image



Refined Topology



Result



Original Image



Binary Image



Refined Topology



Result



ADDITIONAL MATERIAL

Original Image



Binary Image



Refined Topology



Result



